LaunchLens
Complete Project Documentation

Marketing Attribution & Campaign Analytics Platform

Interview and presentation guide based on the repository implementation

This report is grounded in the repository's React/TypeScript frontend, Express/TypeScript backend, Prisma schema, routes, controllers, services, tracking endpoint, and AI provider layer. It explicitly separates implemented behavior from schema-only and future capabilities.

Contents

1. Product overview

LaunchLens is a campaign-link and marketing analytics MVP. Signed-in users create projects and campaigns, share public redirect links, and study recorded click activity through analytics and AI-generated summaries. It is more than a shortener: the redirect is the measurement point that stores a click event before forwarding the visitor.

Problem, users, and solution

Founders and marketing/project owners need a lightweight, owned view of whether campaign links are clicked and which basic traffic characteristics they produce. Common shorteners manage links; full attribution suites can be heavy. LaunchLens focuses on project-scoped campaign links, clicks, browser/device/referrer breakdowns, daily timelines, and concise evidence-grounded AI commentary.

Implemented MVP

• Signup, login, logout, HTTP-only cookie session restoration, and protected UI routes.

• Project CRUD with current-user filtering and duplicate-name checks in application logic.

• Campaign CRUD, unique public slug generation, public redirect tracking links.

• Public click collection: visitor cookie, IP, referrer, UA-derived device/browser, then a 302 redirect.

• Analytics: total clicks, unique browser-cookie visitors, latest click, timeline, devices, browsers, countries, referrers.

• AI insights with Gemini -> Groq -> Mistral fallback and Zod-validated JSON.

Important current limitations

Conversion, source, medium, and campaignTag fields exist in the schema but are not written by campaign controllers; there is no conversion endpoint. Country is stored but no IP-to-country enrichment runs. There is no rate limit, bot detection, cache, queue, or separate TrackingLink model.

Interview pitches

Length	Pitch
30 sec	LaunchLens is a full-stack campaign analytics MVP. Users create campaign redirect links; each public click records a browser-scoped visitor ID, device, browser, referrer, and timestamp before a 302 redirect. React visualizes deterministic Prisma-backed analytics, while a constrained AI layer turns already-calculated metrics into validated recommendations.
1 min	I built LaunchLens to make campaign attribution more explainable than a bare URL shortener. React/Vite uses cookie-backed auth and protected routes. Express validates requests with Zod, stores users/projects/campaigns/clicks via Prisma/PostgreSQL, and enforces ownership through project relationships. The public redirect writes a click event then forwards the visitor. The backend calculates analytics; an AI service builds grounded context and tries Gemini, Groq, then Mistral, accepting only schema-valid JSON.
2 min	LaunchLens is a project-scoped campaign measurement platform. Login sets an HTTP-only JWT cookie. A user creates a Project and Campaign with a destination URL; the server creates a slug and exposes /r/:publicSlug. A visitor opening that link receives or reuses a ll_visitor browser cookie; the server parses User-Agent, records a ClickEvent, and returns a 302. The owner later receives totals, distinct visitor IDs, device/browser/referrer maps and a daily timeline. Those facts feed a provider-abstracted AI service. This is a working MVP; the next steps are bot defense, conversion capture, async ingestion, caching, and geo enrichment.

Resume description: Built LaunchLens, a full-stack campaign attribution MVP that tracks redirect clicks and presents analytics plus grounded AI insights.

• Designed a Prisma/PostgreSQL model for users, projects, campaigns, click events, and optional conversions with cascading relationships.

• Implemented bcrypt hashing, HTTP-only JWT cookies, Zod validation, and owner-scoped access checks.

• Built a public 302 redirect tracker that captures browser-cookie visitor IDs, device/browser data, referrer, and timestamp.

• Created AI provider abstraction with Gemini, Groq, Mistral fallback and strict JSON validation.

2. Architecture and request flow

Client: React 19, TypeScript, Vite, React Router, Axios, React Hook Form/Zod, Tailwind, and Recharts. Server: Node.js/Express 5, TypeScript, Prisma, PostgreSQL, cookie-parser, CORS, bcrypt, jsonwebtoken, ua-parser-js, and AI integrations. Vercel rewrites /api requests to the Render backend.

Architecture diagram

```
Browser
| React SPA / Axios with credentials
v
Vercel: Vite static site + /api rewrite ----> Render: Express API
| auth / projects / campaigns / analytics / AI
| public GET /r/:publicSlug
v
Prisma Client -> Supabase PostgreSQL
|
AI insight request -> Gemini -> Groq -> Mistral (fallback)
```

Layer	Actual responsibility
Pages/components	Forms, dashboard shell, cards/modals, Recharts views, and insight display.
AuthContext/routing	GET /auth/me restores user state; ProtectedRoute waits while auth loads and redirects if needed.
Axios	Uses VITE_API_URL/api locally, /api in production, withCredentials=true.
Express	CORS -> JSON parser -> cookie parser -> route -> auth middleware -> controller -> service/Prisma.
Analytics/AI service	Analytics produces deterministic facts; AI receives only that context and must return validated JSON.
Prisma/PostgreSQL	Typed ORM queries persist relational models and enforce schema constraints.

Complete request lifecycle

```
HTTP -> CORS -> express.json -> cookie-parser -> route
-> authenticate (protected routes) -> controller -> Zod validation / ownership query
-> Prisma query or service -> JSON response
asyncHandler routes Zod errors to errorHandler (400); other errors become 500.
```

Signup, login, and authenticated requests

Signup: POST /api/auth/signup parses name/email/password, checks unique email, hashes at bcrypt salt rounds 10, creates User, returns 201 safe fields. It does not log in. **Login:** validates, finds email, bcrypt-compares, signs { userId } with JWT_SECRET for 7 days, and sets token HTTP-only. For production/HTTPS FRONTEND_URL the cookie is Secure and SameSite=None; otherwise it is Lax. **authenticate:** verifies req.cookies.token and attaches req.user = { userId }; invalid/missing token returns 401.

Project and campaign creation

POST /api/projects validates input, checks same-user project name, writes Project with req.user.userId. POST /api/projects/:projectId/campaigns confirms project ownership, validates name/destinationUrl, creates a normalized slug plus random five-character base-36 suffix, writes Campaign, and returns APP_URL/r/slug.

Tracking and analytics data flow

```
Owner shares APP_URL/r/{publicSlug}
Visitor GET /r/{publicSlug} (public) -> Campaign lookup by unique slug
-> read/create 1-year ll_visitor UUID cookie -> parse User-Agent and referer
-> ClickEvent.create(campaignId, visitorId, req.ip, device, browser, referrer)
-> HTTP 302 campaign.destinationUrl

Owner GET /api/campaigns/:id/analytics -> ownership check -> getCampaignAnalytics -> UI charts
```

The visitor ID identifies a browser cookie, not a human. Repeated clicks by the same browser raise total clicks but normally not unique visitors; deleting cookies or using another browser makes another ID. UAParser data is approximate: Brave may expose a Chrome-like User-Agent. The device helper returns MOBILE for UA containing mobile, TABLET for tablet/iPad, otherwise DESKTOP.

3. Database design and analytics

ER diagram

```
User (1) ----< Project (many) ----< Campaign (many) ----< ClickEvent (many)
ClickEvent (1) ---- (0..1) Conversion

Project.userId -> User.id [CASCADE]
Campaign.projectId -> Project.id [CASCADE]
ClickEvent.campaignId -> Campaign.id [CASCADE]
Conversion.clickEventId -> ClickEvent.id [UNIQUE, CASCADE]
```

Model	Purpose, keys, and relationships
User	cuid PK; name, unique email, bcrypt password, timestamps. One-to-many Projects.
Project	cuid PK; name, optional description/website; userId FK and index; cascade delete. Same-user name uniqueness is controller logic, not a database unique constraint.
Campaign	cuid PK; name, destinationUrl, unique publicSlug; optional source/medium/campaignTag; projectId FK/index; one-to-many ClickEvents. Optional tag fields are currently schema-only.
ClickEvent	cuid PK; campaignId FK/index; visitorId index; optional IP/country/browser/referrer; DeviceType enum; clickedAt. Every redirect writes one.
Conversion	cuid PK; unique clickEventId FK, ConversionType enum, convertedAt. One-to-zero-or-one extension, not currently written by routes.

PostgreSQL suits this ownership-heavy domain because of foreign keys, cascades, indexed lookups, and relational analytics. Prisma exposes typed model operations and generates database queries. The datasource has DATABASE_URL and DIRECT_URL, compatible with a Supabase PostgreSQL deployment.

How analytics are calculated

getCampaignAnalytics runs seven independent operations with Promise.all: campaign metadata; total/unique/latest summary; timeline; devices; browsers; countries; referrers. totalClicks uses count by campaignId. uniqueVisitors uses findMany distinct visitorId. lastClick is latest clickedAt. Devices, browsers, countries, referrers, and daily YYYY-MM-DD timeline are counted using JavaScript Maps after fetching events. Null referrer becomes Direct; null browser/country becomes Unknown. Country will therefore remain Unknown until enrichment is implemented.

4. Backend guide and API reference

Area / key files	What it does
server.ts + app.ts	dotenv, PORT/5000 listener; CORS(FRONTEND_URL, credentials), JSON/cookie middleware, route mounting and error handler.
config/prisma.ts	Exports shared Prisma Client.
routes/*.routes.ts	Maps HTTP endpoints. Project/campaign routers protect all routes; redirect is deliberately public.
auth.controller.ts	Signup/login/me/logout; cookie issuance, clearing, safe profile result.
project.controller.ts	CRUD with current-user filters and duplicate checks.
campaign.controller.ts	Project ownership, slug generation, campaign CRUD, list-time click/visitor counts.
redirect.controller.ts	Slug lookup, visitor cookie, UA parsing, ClickEvent write, 302.
analytics.service.ts	Reusable deterministic aggregation and concurrent combined queries.
ai/*	Context/prompt builders, provider interface/implementations/router, orchestration.
middleware + validation + utils	JWT request guard, generic/Zod error responses, hashes, slugs, visitor IDs, device helper, Zod schemas.

API table

Method / endpoint	Auth	Purpose
POST /api/auth/signup	No	Create user. Body: name, email, password.
POST /api/auth/login	No	Issue JWT cookie. Body: email, password.
GET /api/auth/me; POST /api/auth/logout	Yes	Current safe profile / clear cookie.
GET|POST /api/projects	Yes	List owned projects / create project.
GET|PATCH|DELETE /api/projects/:id	Yes	Read/update/delete owned project.
GET|POST /api/projects/:projectId/campaigns	Yes	List campaign summaries / create campaign.
PATCH|DELETE /api/campaigns/:id	Yes	Update name/destination / delete owned campaign.
GET /r/:publicSlug	No	Record event then 302 redirect.
GET /api/campaigns/:id/analytics	Yes	Return deterministic analytics.
POST /api/campaigns/:id/insights	Yes	Generate grounded AI response.
GET /api/dashboard	Yes	Projects/campaigns/clicks plus five recent projects.

Responses generally use { success, data, message }. Validation errors produce 400; missing/unauthorized protected resources are generally 404 after owner query; auth failures are 401; duplicate email/project names are 409 in those handlers; unhandled controller failures return 500.

5. Frontend guide

The Vite React SPA starts in main.tsx, wrapping BrowserRouter with AuthProvider. AppRoutes exposes Home, Login, Signup, Dashboard, Projects, ProjectDetails, and CampaignAnalytics. ProtectedRoute waits for the first GET /auth/me then sends an unauthenticated user to /login. AuthContext stores user/loading and exposes refreshUser/logout; it restores a valid session after refresh. There is no Redux-style global store in the inspected code.

Page/component group	Purpose and communication
Home and landing components	Hero, Problem, Solution, Workflow, CTA, Navbar, Footer marketing UI.
Login/Signup/auth forms	Forms and validation submit to auth API; login then refreshes global user state.
Dashboard + useDashboard	GET /dashboard drives StatsGrid, RecentProjects and dashboard cards.
Projects + ProjectDetails	Project CRUD; campaign list and create/edit/delete modal interactions call projects/campaigns API.
CampaignAnalytics	Calls analytics endpoint and renders summaries/timeline/device and bar charts; AIInsights invokes insights API.
UI pieces	DashboardShell, Sidebar, Topbar, Card/Button/Loader/EmptyState, campaign/project cards, ChartTooltip.
axios.ts + vercel.json	Local base URL: VITE_API_URL/api. Production: /api, rewritten by Vercel to the Render backend.

```
User action -> page/form -> Axios (withCredentials) -> /api
-> Vercel rewrite in production -> Render Express -> JSON { success, data, message }
-> component state/context -> cards, modals, and Recharts visualizations
```

The frontend includes React Hook Form, resolver, and Zod dependencies plus validation files for auth, project, and campaign forms. Recharts supplies data visualizations; Tailwind is attached through the Vite plugin.

6. Security, deployment, and decisions

Actual security controls

• bcrypt hashes passwords with 10 salt rounds; comparisons happen server-side.

• JWT { userId } expires in seven days and is stored in an HTTP-only token cookie.

• CORS restricts origin to FRONTEND_URL and enables credentials; production chooses Secure + SameSite=None.

• Zod validates untrusted auth/project/campaign input and AI output; Prisma avoids hand-built SQL.

• Owner checks traverse Project/Campaign relationships before protected resource access.

• Secrets are environment variables: database URLs, JWT secret, frontend/app URLs, AI provider keys.

What is not implemented / production hardening

The repository shows no rate limit, CSRF token/origin strategy beyond CORS, CSP/security headers, bot defense, audit log, monitoring, explicit trusted-proxy setup, or explicit secure flag on the visitor cookie. CORS is not a complete CSRF defense. Add rate limiting (especially public redirect and AI), allowlisted destinations, trusted proxy configuration, security headers, CSRF mitigation appropriate to cookie auth, bot heuristics, observability, and secret rotation.

Deployment

frontend/vercel.json rewrites /api/:match* to https://launchlens-backend.onrender.com/api/:match* and all other paths to index.html for SPA navigation. Frontend build: tsc -b && vite build. Backend: tsc then node dist/server.js; PORT is supplied by environment. Browser HTTPS -> Vercel -> Render -> Supabase PostgreSQL, with outbound AI requests. Important caveat: Vercel only rewrites /api in the shown config; APP_URL must point to a host that exposes /r for public redirect links, so verify its live domain configuration.

Decision	Why / trade-off
React + Vite + TS	Fast typed SPA workflow; state orchestration remains component/context based.
PostgreSQL + Prisma	Relationships/cascades/typed queries fit ownership analytics; migrations and relational design add discipline.
JWT HTTP-only cookie	Stateless server verification and XSS-resistant token access; requires careful cookie/CORS/CSRF settings.
Public redirect tracking	Measures at forwarding point; adds a hop and only sees traffic that uses the link.
Deterministic analytics then AI	Reliable counts, LLM explains facts; provider failure/quality still needs validation.

7. AI architecture and technical deep dive

Exact AI pipeline

```
Protected POST /campaigns/:id/insights
-> ownership check -> getCampaignAnalytics
-> buildAIContext (counts and percentages) -> system + user prompts
-> AIProviderRouter: Gemini -> Groq -> Mistral
-> JSON.parse + aiInsightSchema at router -> parse + validate again in service
-> structured response to frontend
```

generateCampaignInsights first ensures campaign.project.userId equals the caller. buildAIContext receives only analytics facts. The system prompt limits analysis to total clicks, unique visitors, traffic sources, devices, browsers, and timeline; it forbids invented conversions, revenue, causes, or user intent; asks cautious small-sample language; and requires one short summary, at most four insights, and at most three recommendations.

AIProvider defines name and generateInsight(systemPrompt, userPrompt). The router creates GeminiProvider, GroqProvider, MistralProvider in that order. A provider error, invalid JSON, or failed Zod validation advances to the next. If all fail, the router throws and the controller returns 500 Unable to generate AI insights. Gemini dynamically uses @google/genai and gemini-2.5-flash; Groq and Mistral use fetch HTTP calls despite their SDK packages being listed. Temperature is 0.2; Groq/Mistral request json_object format.

Technical concepts tied to code

Middleware: a pipeline function; authenticate verifies the cookie before controllers. **REST:** resource URLs with POST/GET/PATCH/DELETE. **async/await:** readable I/O sequencing; **Promise.all:** concurrent independent analytics reads. **TypeScript interfaces:** analytics.ts describes data contracts; AIProvider makes providers interchangeable. **Zod:** validates incoming and model-generated JSON. **ORM:** Prisma maps typed model queries to relational database operations. **Environment variables:** keep connection strings, JWT secret, and provider keys out of source.

Why not let the LLM calculate analytics?

Database/service calculations are deterministic, reproducible, and testable. The model is used for concise interpretation only. Grounding rules, low temperature, JSON-only output, provider-level validation, and service-level validation reduce - but do not eliminate - output risk.

8. Interview preparation, roadmap, and cheat sheet

Representative request payloads

```
POST /api/auth/signup
{ "name":"Asha Rao", "email":"asha@example.com", "password":"securepass" }
POST /api/projects
{ "name":"Product Launch", "description":"Q4 launch", "website":"https://example.com" }
POST /api/projects/{projectId}/campaigns
{ "name":"LinkedIn launch", "destinationUrl":"https://example.com/launch" }
Response includes publicSlug and trackingLink: APP_URL/r/linkedin-launch-abc12
```

Question	Interview-ready answer	Follow-up answer
Is it just Bitly?	It uses a redirect like a shortener, but its core is ownership, persisted events, analytics, and constrained AI interpretation.	MVP attribution means a click is credited to the Campaign slug used; it is not multi-touch or conversion/revenue attribution yet.
How identify unique visitors?	A one-year ll_visitor HTTP-only UUID cookie; analytics counts distinct IDs per campaign.	It is browser-level, not person-level; another browser or cleared cookies creates another identity.
What if someone clicks multiple times?	Every redirect writes a click, so total clicks grows; same persistent cookie normally remains one unique visitor.	This distinguishes activity volume from browser reach.
Why can Brave show as Chrome?	UAParser interprets the User-Agent; privacy browsers may expose a Chrome-compatible UA.	It is an inference, not definitive browser identity.
Why PostgreSQL/Prisma?	Clear owner-resource relationships, FK cascades, indexes, and typed queries.	MongoDB is possible, but relational modeling fits the queries and integrity needs.
If Gemini fails?	Router catches the failure/invalid JSON, then tries Groq and Mistral.	All fail: service throws; controller returns 500 unavailable insight response.
How scale clicks?	Use edge/queue ingestion, batch writes, group/rollup tables, cache reads, Redis rate limits, background processing.	Keep redirect path very fast; define durable async delivery semantics.
Biggest weakness?	MVP aggregation is in memory and lacks bots, conversion capture, and geo enrichment.	The schema gives clear extension points, but I would not claim those features are already complete.

Five-minute presentation script

• Start with the need: campaign links should produce explainable evidence, not only redirects.

• Show the solution: project -> campaign -> public link -> click event -> dashboard.

• Draw the architecture: React/Vercel -> Express/Render -> Prisma/Supabase; public redirect is the key event boundary.

• Explain auth and ownership filtering, then the browser-cookie visitor trade-off.

• Walk through deterministic aggregation and visible charts.

• Explain AI as constrained interpretation and provider fallback, not magic analytics.

• Close with honest V2: rate limit/bots, conversions, queues/caches, geo enrichment, monitoring.

Strongest technical highlights

Rank	Highlight	Why
1	End-to-end tracking flow	Public slug becomes a real ClickEvent then a 302, feeding analytics.
2	AI provider abstraction/fallback	Interface, ordered router, JSON parsing and Zod validation are concrete reliability controls.
3	Ownership-aware API	User -> Project -> Campaign is checked for protected access.
4	Session restoration	AuthContext invokes /me at startup and routes wait for its result.
5	Typed relational model	Enums, unique constraints, indexes, foreign keys, and cascade deletion.

Current vs future

Current implementation	Future improvement
Synchronous event insert and in-memory Map aggregation	Queue/batch ingestion, grouped SQL/rollups, Redis/cache, warehouse analytics.
Cookie + User-Agent basic attribution	Consent-aware identities, bot filtering, IP/geo enrichment, normalized referrers.
Conversion schema only	Conversion pixel/webhook, attribution rules, revenue reports.
On-demand AI	Observability, cost controls, caching, evaluations, retries and feedback.
Basic auth/CORS/bcrypt	Rate limits, CSRF design, security headers, audits, monitoring.

Final cheat sheet

Stack: React, TypeScript, Vite, Tailwind, Axios, Router, Recharts; Express, Prisma, PostgreSQL, Zod, bcrypt, JWT, UAParser; Gemini/Groq/Mistral. **Models:** User -> Project -> Campaign -> ClickEvent -> optional Conversion. **Auth:** 7-day HTTP-only JWT cookie, middleware sets req.user. **Tracking:** GET /r/slug -> visitor cookie + UA/referrer/IP -> ClickEvent -> 302. **Analytics:** count, distinct visitor, Maps grouped by device/browser/country/referrer/day. **AI:** facts -> grounded prompts -> fallback -> JSON/Zod. **Do not overclaim:** no wired conversion capture, geo enrichment, bot protection, rate limiting, CSRF design, or separate TrackingLink model.

Do not say the MVP has real conversion attribution, geographic analytics, UTM capture, or bot protection unless you clearly frame those as planned. The strongest interview story is an honest working MVP with a defensible end-to-end data flow and clear production upgrade path.